

Trabajo Práctico 3

Tema: Arquitectura, Escalabilidad y Gestores NoSQL

Alumno: Ignacio Figueroa

Comisión: 7

Actividad 1: Solución de Infraestructura (Caso: "Ticket-All")

1. Escalabilidad Vertical vs. Horizontal La idea de la empresa de comprar más RAM es lo que llamamos **Escalabilidad Vertical**. El problema es que esto tiene un "techo": por más plata que tengas, llega un punto en que no existen servidores con tanta memoria, o son carísimos y no rinden lo que valen. Además, es peligroso; si ese servidor superpotente tiene un problema técnico, la página se cae por completo y nadie compra nada.

Para un caso como "Ticket-All", conviene la **Escalabilidad Horizontal**. En vez de un solo servidor gigante, usamos muchos servidores normales trabajando en equipo. Es mejor porque podés ir sumando máquinas a medida que se conecta más gente y, si una falla, las otras siguen funcionando, por lo que la venta no se corta

2. Sharding y el "Punto Caliente" Para que los miles de compradores no saturen una sola máquina, usaría una arquitectura de **Sharding**. Como **Shard Key**, elegiría el `ID_usuario`. Esto sirve para que los usuarios se repartan de forma pareja entre todos los servidores disponibles.

Si eligiéramos, por ejemplo, el "Nombre del Artista" como clave, tendríamos un **"Punto Caliente" (Hotspot)**. Esto pasa cuando casi todo el tráfico va a parar a un solo servidor (porque todos quieren ver al mismo artista) mientras los otros servidores están al vicio. Usando el ID de usuario, evitamos esto porque la carga se distribuye equitativamente sin importar a qué show quieran ir.

3. Replicación (Replica Sets) La replicación es como tener "auxiliares" listos para saltar a la cancha. Si el servidor que tiene los datos falla, el sistema no se frena gracias a los diferentes roles:

- **Nodo Primario:** Es el encargado de recibir todas las compras y cambios en los datos en tiempo real.
- **Nodos Secundarios:** Son copias exactas del primario. Si el Primario se cae por la cantidad de gente, uno de los Secundarios toma su lugar automáticamente y la venta sigue como si nada.

Actividad 2: Modelado y Consistencia (Caso: "Fintech-Plus")

1. ACID vs. BASE Para esta aplicación usaría los dos modelos dependiendo de qué estemos guardando:

- **Tipo A (Salos y transferencias):** Acá usaría **ACID**. Necesitamos **Consistencia Inmediata**. Si yo te transfiero 1000 pesos, ese monto se tiene que descontar de mi cuenta y aparecer en la tuya al mismo tiempo. No puede haber un "gris" donde la plata no esté en ningún lado, porque sería un desastre contable.
- **Tipo B (Muro de noticias y likes):** Para esto aplicaría **BASE**. Acá usamos **Consistencia Eventual**. Si vos le das "like" a una noticia y yo tardo dos segundos más en verlo, no pasa nada. Lo importante es que el sistema sea rápido y esté siempre disponible, aunque los datos tarden un poquito en actualizarse para todos los usuarios.

2. Optimización de consultas: Desnormalización En las bases de datos relacionales (SQL), solemos tener los comentarios en una tabla y los usuarios en otra, y los unimos con un JOIN. Pero en NoSQL (como MongoDB), los JOINS son lentos y se evitan.

Para resolver esto usaría la **Desnormalización**. Básicamente, en lugar de guardar solo el ID_Usuario en el comentario, guardo directamente el Nombre y la URL_Foto adentro del documento del comentario.

- **Ventaja:** Cuando la app quiere mostrar el muro, hace una sola consulta y ya tiene toda la info para dibujar la pantalla. No tiene que andar saltando de una tabla a otra.
- **Desventaja:** Si el usuario cambia su foto, tengo que actualizarla en todos sus comentarios, pero como la gente cambia la foto una vez por año y lee comentarios todo el día, el beneficio de velocidad vale la pena.

. Flujo de datos y componentes de MongoDB Cuando un usuario abre la app y pide ver el muro, la consulta hace este camino:

1. **Mongos (Query Router):** Es el "recepcionista". La app le pregunta al Mongos: "¿Dónde están los comentarios de hoy?". Él no tiene los datos, pero sabe a quién preguntarle.
2. **Config Servers:** Son el "mapa" del sistema. El Mongos les consulta a ellos para saber en qué fragmento (shard) de la base de datos está guardada la info que buscamos.

3. **Shards:** Una vez que el Mongos sabe a dónde ir, busca la info en el servidor correspondiente, junta los resultados y se los devuelve a la app del usuario.

Actividad 3: Investigación

1. Teorema CAP y Principio BASE El **Teorema CAP** dice que en un sistema distribuido es imposible cumplir al mismo tiempo con las tres características: **Consistencia** (todos ven lo mismo), **Availability/Disponibilidad** (el sistema siempre responde) y **Particionamiento** (el sistema sigue andando si se corta la red entre servidores).

Como en internet los fallos de red (Particiones) son inevitables, tenemos que elegir: o el sistema es Consistente o está Disponible. Ahí entra el **principio BASE**, que es el "opuesto" a ACID. BASE prefiere que el sistema esté siempre andando (Disponibilidad) aunque los datos no sean exactos al segundo, confiando en que eventualmente todo se va a sincronizar.

- **Fuente consultada:** *Documentación oficial de IBM sobre Teorema CAP.*

2. Redis vs. MongoDB para Leaderboards Redis es mucho más rápido para una tabla de posiciones porque trabaja **100% en memoria RAM**, mientras que MongoDB suele tener que ir al disco. Además, Redis tiene estructuras de datos específicas para esto, como los "Sorted Sets".

Comandos de Redis:

- **INCR:** Suma 1 a un valor (ideal para contar votos o puntos).
- **LPUSH:** Agrega un elemento al principio de una lista.
- **SADD:** Agrega un elemento a un conjunto (evitando que se repita).
- **Fuente consultada:** *Redis.io (Documentación oficial).*

3. Neo4j y Redes de Fraude Empresas como Neo4j dicen que son mejores que MongoDB para detectar fraudes porque su estructura de **nodos y aristas** está pensada para ver conexiones. En MongoDB, para ver si un estafador está conectado con 5 cuentas bancarias y 3 direcciones diferentes, tendrías que hacer muchas consultas o documentos muy complejos. En Neo4j, las relaciones son "ciudadanos de primera clase"; podés saltar de un dato a otro siguiendo las flechas de forma instantánea. Esto permite encontrar patrones sospechosos (como una "red" de personas usando el mismo teléfono) mucho más rápido.

- **Fuente consultada:** *Neo4j Graph Data Platform - "Fraud Detection with Graph Databases".*

